

Data Definition Language (DDL):

1. Create a table named "Students" with columns: StudentID (int), FirstName (varchar), LastName (varchar), and Age (int).

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY,  
    FirstName VARCHAR(50),  
    LastName VARCHAR(50),  
    Age INT  
);
```

2. Add a new column "Gender" of type varchar to the "Students" table.

```
ALTER TABLE Students  
ADD Gender VARCHAR(10);
```

3. Rename the "Age" column to "StudentAge" in the "Students" table.

```
ALTER TABLE Students  
RENAME COLUMN Age TO StudentAge;
```

4. Drop the "Gender" column from the "Students" table.

```
ALTER TABLE Students  
DROP COLUMN Gender;
```

Data Manipulation Language (DML):

1. Insert a new student with StudentID 1, FirstName "John", LastName "Doe", Age 25 into the "Students" table.

```
INSERT INTO Students (StudentID, FirstName, LastName, StudentAge)  
VALUES (1, 'John', 'Doe', 25);
```

2. Update the age of the student with StudentID 1 to 26.

```
UPDATE Students  
SET StudentAge = 26
```

```
WHERE StudentID = 1;
```

3. Delete the student with StudentID 1 from the "Students" table.

```
DELETE FROM Students  
WHERE StudentID = 1;
```

4. Insert multiple students into the "Students" table in a single query.

```
INSERT INTO Students (StudentID, FirstName, LastName, StudentAge)  
VALUES  
(1, 'John', 'Doe', 25),  
(2, 'Sarah', 'Smith', 22),  
(3, 'David', 'Brown', 19),  
(4, 'Anna', 'Stone', 28),  
(5, 'Mike', 'Jones', 17);
```

5. Update the LastName of all students whose FirstName is "John" to "Smith".

```
UPDATE Students  
SET LastName = 'Smith'  
WHERE FirstName = 'John';
```

6. Delete all students whose age is less than 18.

```
DELETE FROM Students  
WHERE StudentAge < 18;
```

7. Insert a new course with CourseID 1, CourseName "Mathematics", and Instructor "Professor X" into the "Courses" table.

```
INSERT INTO Courses (CourseID, CourseName, Instructor)  
VALUES (1, 'Mathematics', 'Professor X');
```

8. Update the Instructor of the course with CourseID 1 to "Professor Y".

```
UPDATE Courses  
SET Instructor = 'Professor Y'
```

```
WHERE CourseID = 1;
```

9. Delete the course with CourseID 1 from the "Courses" table.

```
DELETE FROM Courses
```

```
WHERE CourseID = 1;
```

10. Truncate the data in the "Students" table.

```
TRUNCATE TABLE Students;
```

11. Drop the "Courses" table.

```
DROP TABLE Courses;
```

Retrieval:

1. Retrieve all columns from the "Students" table.

```
SELECT *
```

```
FROM Students;
```

2. Retrieve only the FirstName and LastName columns from the "Students" table.

```
SELECT FirstName, LastName
```

```
FROM Students;
```

3. Retrieve distinct values of the LastName column from the "Students" table.

```
SELECT DISTINCT LastName
```

```
FROM Students;
```

4. Retrieve all students aged 20 or above.

```
SELECT *
```

```
FROM Students
```

```
WHERE StudentAge >= 20;
```

5. Retrieve all students whose last name starts with 'S'.

```
SELECT *
```

```
FROM Students  
WHERE LastName LIKE 'S%';
```

6. Retrieve all students whose first name contains the letter 'a'.

```
SELECT *  
FROM Students  
WHERE FirstName LIKE '%a%';
```

7. Retrieve the oldest student from the "Students" table.

```
SELECT *  
FROM Students  
ORDER BY StudentAge DESC  
LIMIT 1;
```

8. Retrieve the count of students in the "Students" table.

```
SELECT COUNT(*) AS Total_Students  
FROM Students;
```

9. Retrieve the total number of students grouped by age.

```
SELECT StudentAge, COUNT(*) AS Total_Students  
FROM Students  
GROUP BY StudentAge;
```

10. Retrieve the average age of students.

```
SELECT AVG(StudentAge) AS Average_Age  
FROM Students;
```

11. Retrieve all students sorted by age in ascending order.

```
SELECT *  
FROM Students  
ORDER BY StudentAge ASC;
```

12. Retrieve all students sorted by age in descending order.

```
SELECT *
```

```
FROM Students  
ORDER BY StudentAge DESC;
```

13.Retrieve the top 5 oldest students.

```
SELECT *  
FROM Students  
ORDER BY StudentAge DESC  
LIMIT 5;
```

14.Retrieve the 10 youngest students.

```
SELECT *  
FROM Students  
ORDER BY StudentAge ASC  
LIMIT 10;
```

15.Retrieve the students aged between 20 and 30.

```
SELECT *  
FROM Students  
WHERE StudentAge BETWEEN 20 AND 30;
```

16. Retrieve the count of students.

```
SELECT COUNT(*) AS Total_Students  
FROM Students;
```

17.Retrieve the total number of courses.

```
SELECT COUNT(*) AS Total_Courses  
FROM Courses;
```

18.Retrieve the average age of students.

```
SELECT AVG(StudentAge) AS Average_Age  
FROM Students;
```

19.Retrieve the maximum age of students.

```
SELECT MAX(StudentAge) AS Maximum_Age
```

```
FROM Students;
```

20.Retrieve the minimum age of students.

```
SELECT MIN(StudentAge) AS Minimum_Age  
FROM Students;
```

21.Retrieve the count of students grouped by age.

```
SELECT StudentAge, COUNT(*) AS Total_Students  
FROM Students  
GROUP BY StudentAge;
```

22.Retrieve the number of courses taught by each instructor.

```
SELECT Instructor, COUNT(*) AS Total_Courses  
FROM Courses  
GROUP BY Instructor;
```

23.Retrieve the average age of students enrolled in each course.

```
CREATE TABLE StudentCourses (  
    StudentID INT,  
    CourseID INT,  
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),  
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)  
);  
  
SELECT c.CourseName,  
       AVG(s.StudentAge) AS Average_Age  
FROM Courses c  
JOIN StudentCourses sc ON c.CourseID = sc.CourseID  
JOIN Students s ON sc.StudentID = s.StudentID  
GROUP BY c.CourseID, c.CourseName;
```

24.Retrieve the oldest student enrolled in each course.

```
SELECT c.CourseName,  
       MAX(s.StudentAge) AS Oldest_Age  
FROM Courses c  
JOIN StudentCourses sc ON c.CourseID = sc.CourseID  
JOIN Students s ON sc.StudentID = s.StudentID  
GROUP BY c.CourseID, c.CourseName;
```

25.Retrieve the youngest student enrolled in each course.

```
SELECT c.CourseName,  
       MIN(s.StudentAge) AS Youngest_Age  
FROM Courses c  
JOIN StudentCourses sc ON c.CourseID = sc.CourseID  
JOIN Students s ON sc.StudentID = s.StudentID  
GROUP BY c.CourseID, c.CourseName;
```

26. Retrieve the number of male and female students.

```
SELECT Gender, COUNT(*) AS Total_Students  
FROM Students  
GROUP BY Gender;
```

27.Retrieve the count of students whose age is less than 25.

```
SELECT COUNT(*) AS Total_Students  
FROM Students  
WHERE StudentAge < 25;
```

28.Retrieve the count of students whose age is between 20 and 30

```
SELECT COUNT(*) AS Total_Students  
FROM Students  
WHERE StudentAge BETWEEN 20 AND 30;
```

29.Retrieve the average age of students whose last name starts with 'S'.

```
SELECT AVG(StudentAge) AS Average_Age  
FROM Students  
WHERE LastName LIKE 'S%';
```

30. Retrieve the count of students whose first name contains the letter 'a'

```
SELECT COUNT(*) AS Total_Students  
FROM Students  
WHERE FirstName LIKE '%a%';
```